FRONTEND IMPLEMENTATION: ENABLING DYNAMIC CONFIGURATION

On the frontend, the `useChat` hook simplifies this by allowing us to pass additional data to the API using its body parameter. We use React's `useState` hook to manage the user's selected provider and model and then pass these values directly to the `useChat` hook.

Listing 3.13 Updating page.js component to allow multiple providers

```
const providers = [.
  { value: 'openai', label: 'OpenAI' },
  { value: 'gemini', label: 'Google AI' },
];

const models = {.
  openai: [
    { value: 'gpt-3.5-turbo', label: 'GPT-3.5-turbo' },
    { value: 'gpt-4', label: 'GPT-4' },
  ],
  gemini: [{ value: 'models/gemini-2.0-flash', label: 'Gemini' }],
};

const Chat = () => {
  const [provider, setProvider] =
            React.useState('openai');
  const [model, setModel] = React.useState('gpt-4');

  const { messages, input, handleInputChange, handleSubmit,
  isLoading } = useChat({
    api: '/api',
    body: { provider, model },
  });
```

The providers object has a value property (used as the identifier) and a label property (used for display purposes).

Each model is represented by an object with a value property (used as the identifier) and a label property (used for display purposes).

We use the useState hook to manage the selected provider and model.

We pass the selected provider and model values to the body parameter of the useChat hook.

We can verify that this change works by starting the development server and using the `<Dropdown/>` component to choose the model we currently use. Since the `useChat` hook manages the list of messages in a unified way, we can literally swap providers on the fly while passing the context to each one of them. Figure 3.6 shows a screenshot of the application after a few interactions with both models.

Feel free to test out different providers and compare their responses. With this functionality in place, you can explore the potential of integrating your own custom-made LLM in the future. This approach allows you to have greater control over the performance and safety of the generated responses.

As we move forward, we will take a look at the world of multimedia content and how it can greatly enhance the user experience in conversational UIs.

3.4 Enhancing conversational UIs with multimedia content

Building a truly engaging conversational UI means moving beyond text. Multimodality, the ability to understand and generate content using multiple modes of input like images, audio, and video, is a crucial next step.